CMR TECHNICAL CAMPUS

INDUSTRIAL ORIENTED MINI PROJECT - 2026

Detecting and Mitigating Botnet Attacks in Software-Defined Networks Using Deep Learning Techniques

Guide :

A.Divya Barati

Associate Professor

Presenting By:-

Batch F16

U.Varshitha-247R1A05CG

T.Sai Lalith Chandra-247R1A05CF

D.SHASHI KUMAR-- 247R1A05AD

Table of Contents

1. ABSTRACT
2. EXISTING SYSTEM
3. DISADVANATGES
4. PROPOSED SYSTEM
5. ADVANATGES
6. HARDWARE AND SOFTWARE REQUIREMENTS
7. NOVELTY OF THE PROJECT
8. ARCHITECTURE OF THE SYSTEM
9. MODULES OF THE SYSTEM
10. CLASS DIAGRAM
11. USECASE DIAGRAM
12. SEQUENCE DIAGRAM
13. COLLABORATION DIADRAM
14. SAMPLE CODE
15. SAMPLE OUTPUTS
16. CONCLUSION
17. REFERENCES

1. ABSTRACT..

•Software Defined Networking (SDN) is a modern network architecture that separates the **control plane and data plane**.

•It allows centralized network management and flexible configuration.

•However, SDN networks are vulnerable to **cyber attacks like Botnet and DDoS attacks**.

•Botnets are groups of compromised devices controlled by attackers.

•These attacks can overload the SDN controller and disrupt network services.

•This project proposes a **Deep Learning based system** to detect and mitigate botnet attacks.

•The system analyzes network traffic and identifies malicious patterns using machine learning and deep learning models.

2.Existing System

•Traditional intrusion detection systems use **Machine Learning algorithms**.

•Common algorithms used:

- Decision Tree
- Support Vector Machine (SVM)
- Random Forest

•Some research uses **Deep Learning models** like RNN and LSTM.

•These systems detect attacks by analyzing network traffic data.

•Detection depends on feature extraction and dataset quality.

3. Disadvantages

- Difficult to process **large and complex network datasets**.
- Requires **large amount of labeled training data**.
- Incorrect data labeling reduces accuracy.
- High computational time in some algorithms.
- Limited accuracy in detecting advanced botnet attacks.

4. Proposed System

•The proposed system uses **Deep Learning techniques** for botnet detection.

•Network traffic data is collected and preprocessed.

•Important features are selected using **feature weighting and tuning techniques**.

•Multiple algorithms are used for comparison:

- RNN
- CNN
- LSTM
- DNN
- MLP

•The system evaluates model performance using metrics like accuracy, precision, recall, and F1-score.

•Among all models, **CNN provides the best detection performance**.

5. Advantages

- High detection accuracy.
- Faster detection of malicious traffic.
- Efficient feature selection improves performance.
- Handles large datasets effectively.
- Reduces controller overload in SDN networks.
- Improves overall network security.

6. Hardware & Software Requirements

Hardware Requirements

- Processor : Intel i3 / Pentium IV
- RAM : Minimum 4 GB
- Hard Disk : Minimum 20 GB
- Keyboard & Mouse
- Monitor : SVGA

Software Requirements

- Operating System : Windows 7 / Windows 10
- Programming Language : Python
- Framework : Django ORM
- Frontend : HTML, CSS, JavaScript
- Database : MySQL
- Tool : Jupyter Notebook

7. Novelty of the Project

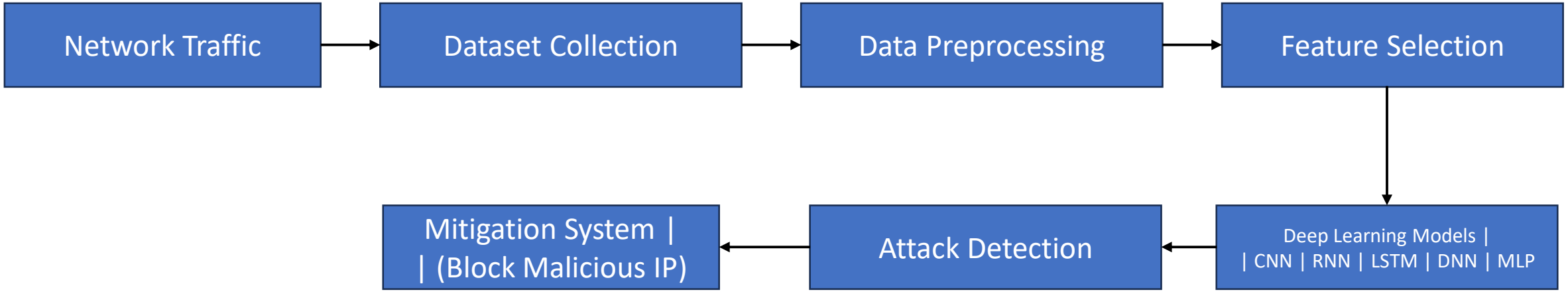
- Uses **Deep Learning models for SDN security**.
- Implements **feature optimization techniques**.
- Compares multiple machine learning and deep learning models.
- Uses **CNN model for high accuracy attack detection**.
- Provides **efficient mitigation mechanism using dynamic flow deletion**.
- Achieves detection accuracy of **around 99%**.

8. Architecture of the System

System Architecture Components:

1. Data Collection Module
2. Data Preprocessing Module
3. Feature Selection Module
4. Machine Learning / Deep Learning Models
5. Attack Detection Module
6. Mitigation Module

Flow:



9. Modules of the System

Main Modules:

1. Dataset Upload

•Upload the network traffic dataset.

2. Dataset Preprocessing

•Cleaning the dataset.

•Converting non-numeric values to numeric values.

•Handling imbalance using SMOTE.

3. Model Training

•Training machine learning and deep learning algorithms.

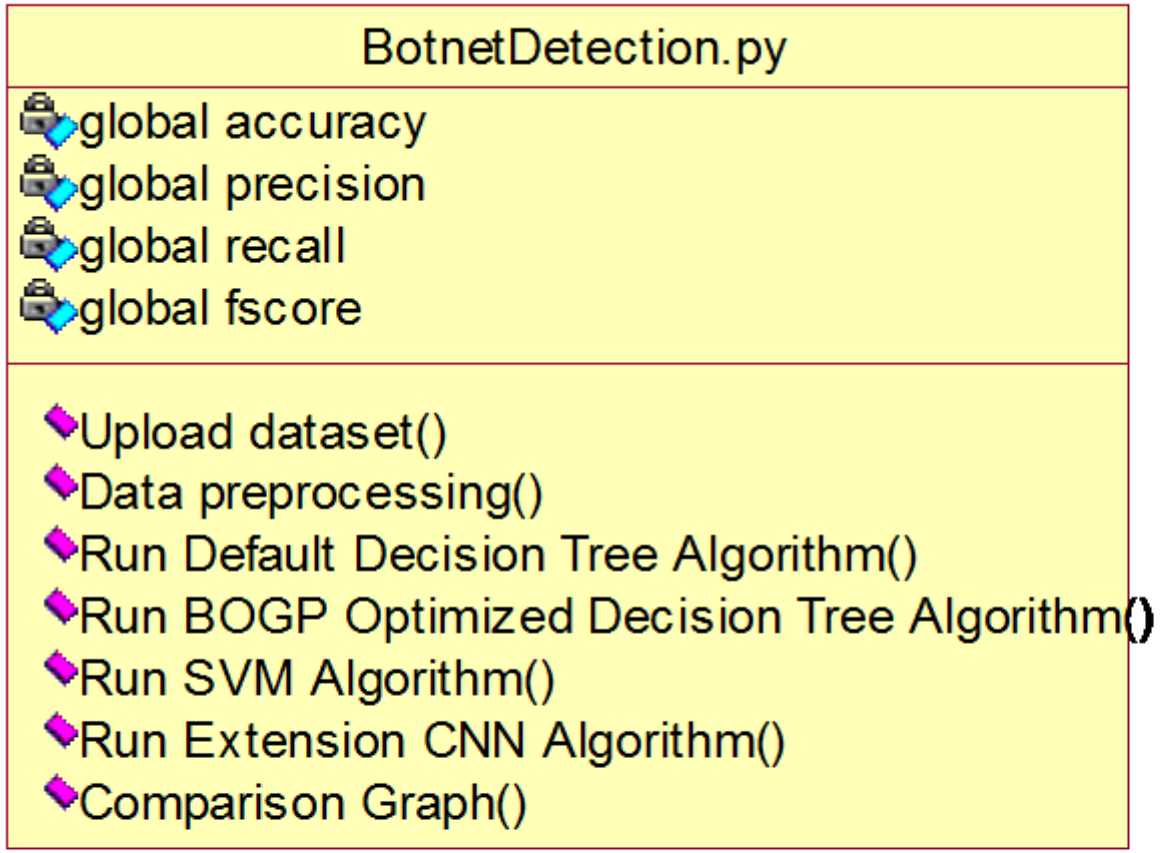
4. Attack Detection

•Classifying traffic as **Normal or Attack**.

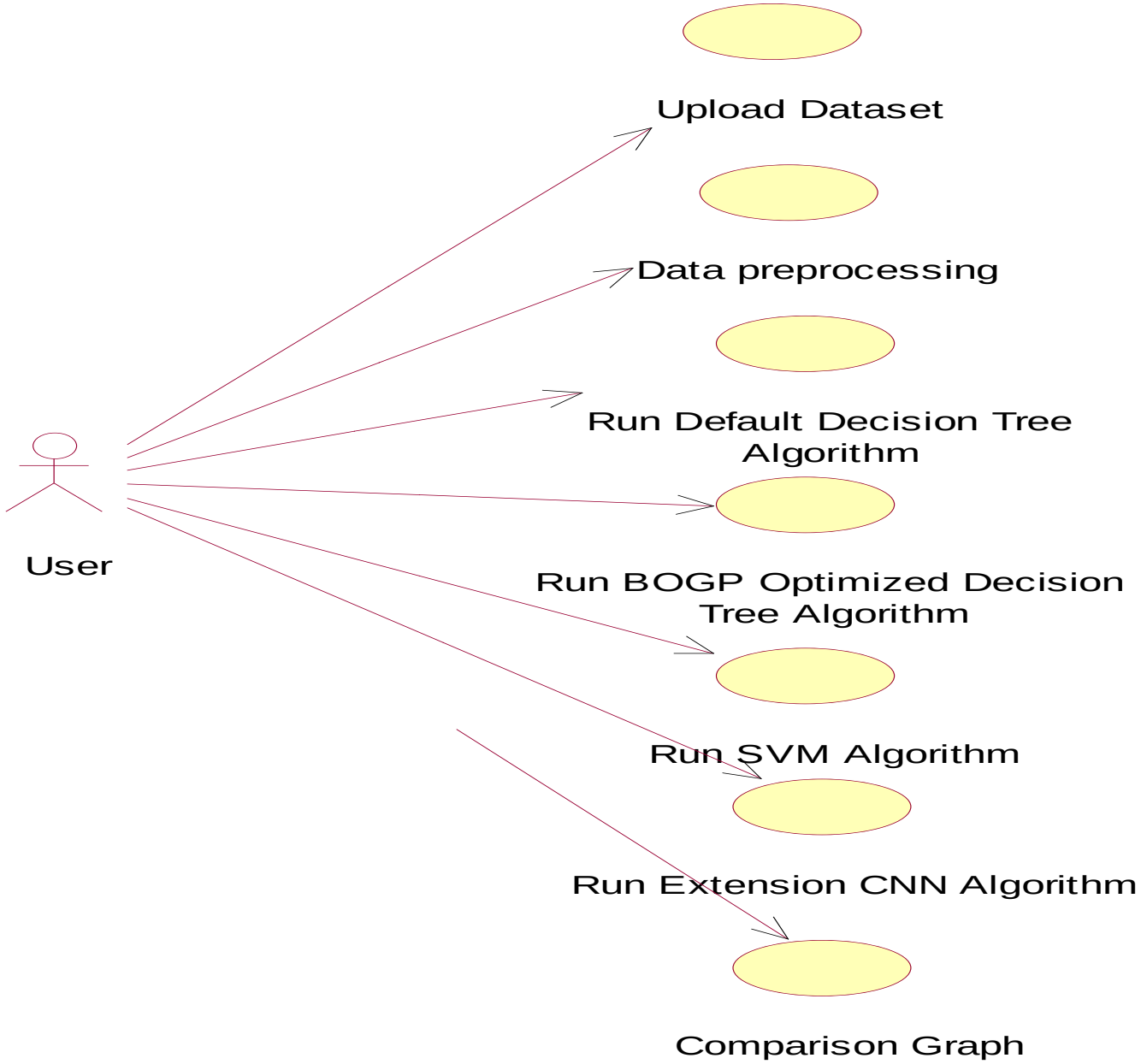
5. Performance Analysis

•Comparing algorithms using accuracy, precision, recall and F1 score.

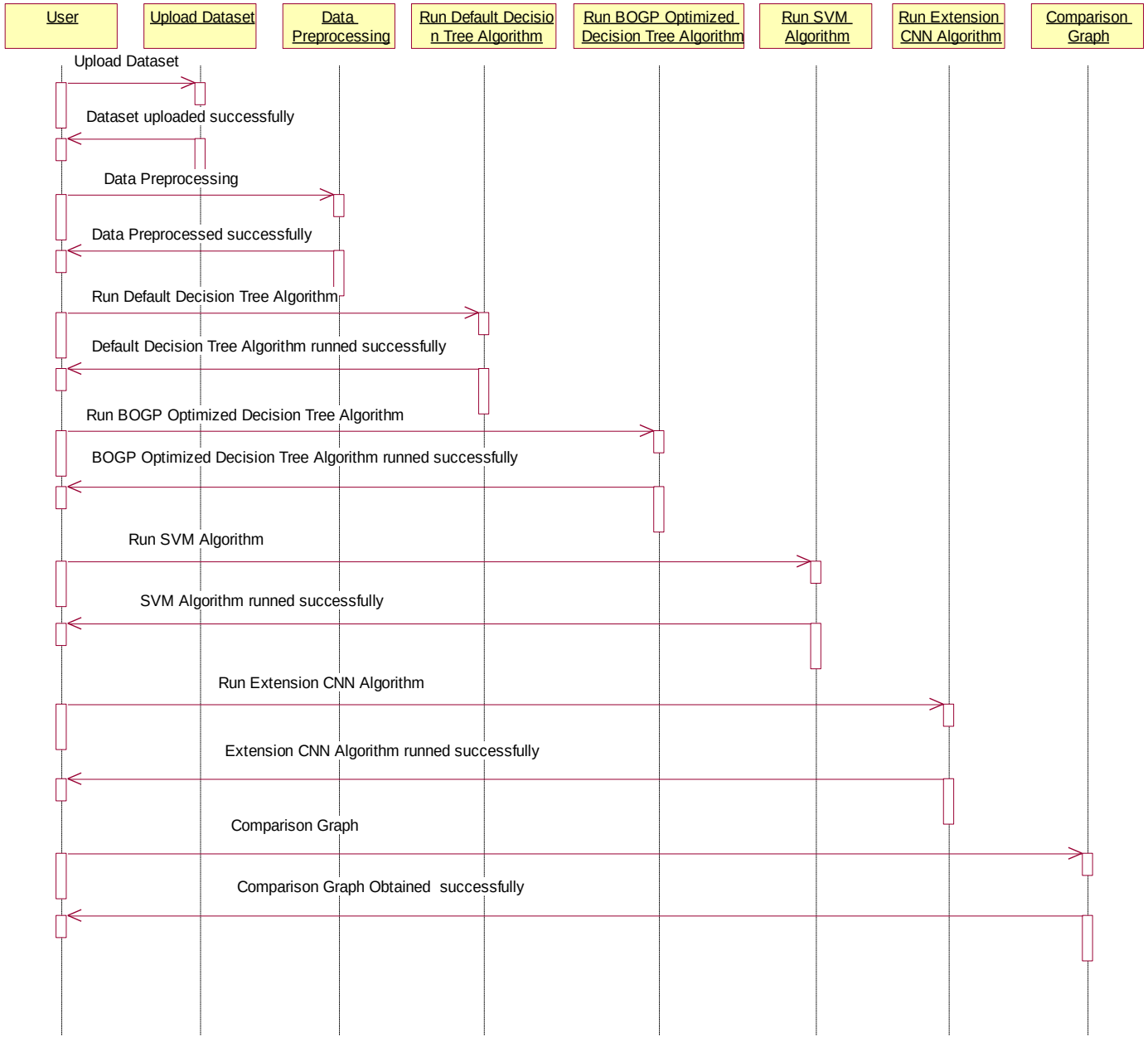
10.CLASS DIAGRAM:



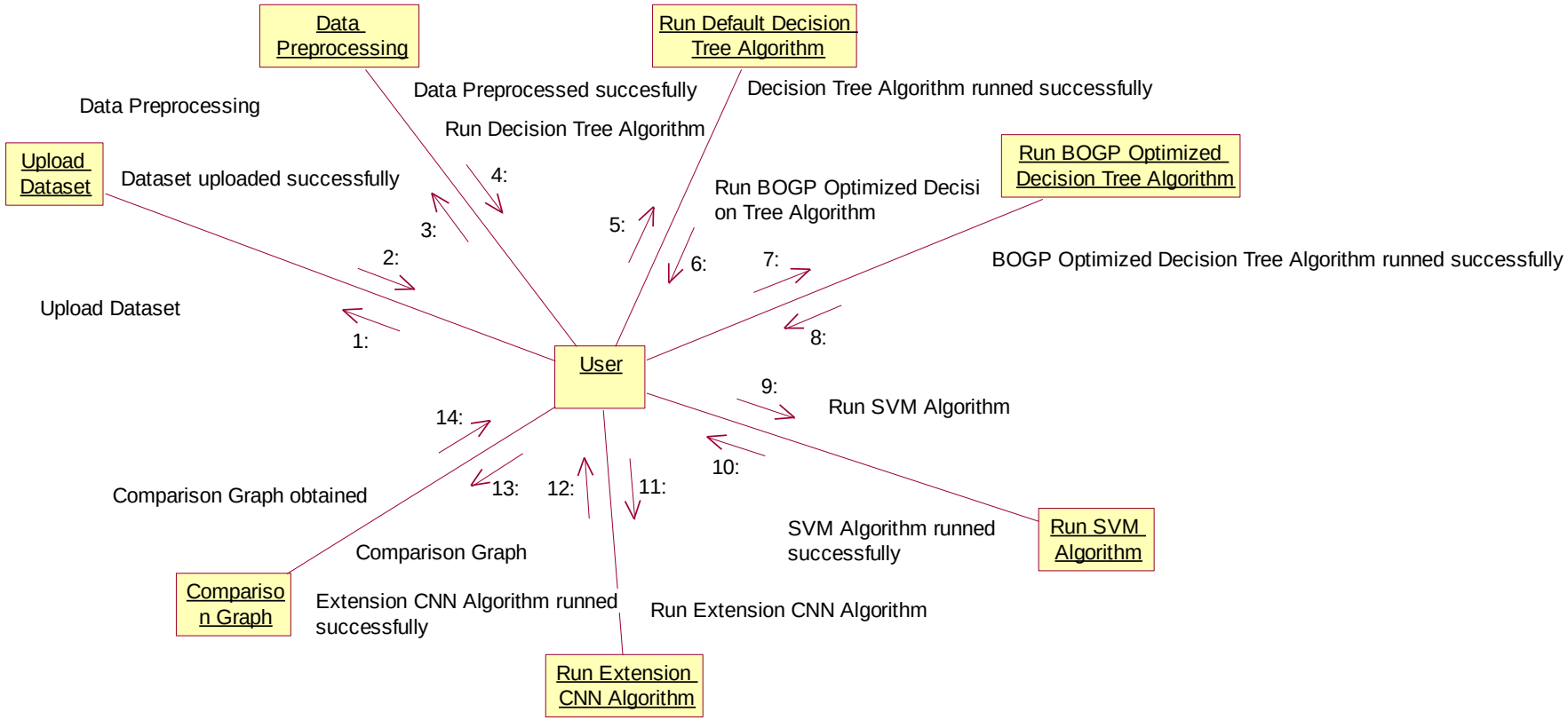
11.USECASE DIAGRAM:



12.SEQUENCE DIAGRAM:



13.COLLABORATION DIAGRAM:



14.Sample Code:

```
import pandas as pd
import numpy as np
from bayes_opt import BayesianOptimization #importing byaseian optimization class
import matplotlib. pyplot as plt #use to visualize dataset values
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import f1_score
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn import metrics
from sklearn.preprocessing import LabelEncoder
from sklearn import svm
import os
from keras.utils.np_utils import to_categorical
from keras.layers import  MaxPooling2D
from sklearn import svm
import os
from keras.utils.np_utils import to_categorical
from keras.layers import  MaxPooling2D
from keras.layers import Dense, Dropout, Activation, Flatten, GlobalAveragePooling2D, BatchNormalization
from keras.layers import Convolution2D
from keras.models import Sequential
from keras.callbacks import ModelCheckpoint
import pickle
dataset = pd.read_csv("Dataset/UNSW_2018_IoT_Botnet_Full5pc_4.csv", low_memory=False)
dataset
labels, count = np.unique(dataset['attack'].ravel(), return_counts = True)
print("Normal Records : "+str(count[0]))
print("Attack Records : "+str(count[1]))
height = count
bars = labels
y_pos = np.arange(len(bars))
plt.figure(figsize = (4, 3))
plt.bar(y_pos, height)
```

```
plt.xticks(y_pos, bars)
plt.xlabel("Dataset Class Label Graph")
plt.ylabel("Count")
plt.xticks(rotation=90)
plt.show()
dataset.isnull().sum()
dataset.groupby("proto").size().plot.pie(autopct='%.0f%%',
figsize=(6, 6))
plt.title("Different Protocol Graph")
plt.xlabel("Protocol Name")
plt.ylabel("Usage %")
plt.show()
data = dataset[['proto', 'attack']]
plt.figure(figsize=(6,4))
sns.boxplot(data=data, x='proto', y='attack', palette='rainbow')
plt.title("Different Protocols Used by Attacks")
plt.show()
dataset.drop(['pkSeqID', 'category', 'subcategory'], axis =
1,inplace=True)
#using label encoder to convert non-numeric values to numeric
values
encoder1 = LabelEncoder()
encoder2 = LabelEncoder()
encoder3 = LabelEncoder()
encoder4 = LabelEncoder()
encoder5 = LabelEncoder()
encoder6 = LabelEncoder()
encoder7 = LabelEncoder()
dataset['flgs'] =
pd.Series(encoder1.fit_transform(dataset['flgs'].astype(str)))#encode
all str columns to numeric
dataset['proto'] =
pd.Series(encoder2.fit_transform(dataset['proto'].astype(str)))#encod
e all str columns to numeric
dataset['saddr'] =
pd.Series(encoder3.fit_transform(dataset['saddr'].astype(str)))#encod
e all str columns to numeric
dataset['sport'] =
pd.Series(encoder4.fit_transform(dataset['sport'].astype(str)))#encod
e all str columns to numeric
dataset['daddr'] =
pd.Series(encoder5.fit_transform(dataset['daddr'].astype(str)))#encod
e all str columns to numeric
dataset['dport'] =
pd.Series(encoder6.fit_transform(dataset['dport'].astype(str)))#encod
e all str columns to numeric
```

```
dataset['state'] = pd.Series(encoder7.fit_transform(dataset['state'].astype(str)))#encode all str columns to numeric
Dataset
pca = PCA(n_components=2)
temp = dataset.values
X = temp[:,0:temp.shape[1]-1]
Y = temp[:,temp.shape[1]-1]
X = pca.fit_transform(X)
plt.figure(figsize=(6,4))
plt.scatter(X[:, 0], X[:, 1], c=Y)
plt.xlabel("First Principal Component")
plt.ylabel("Second Principal Component")
plt.title("Dataset Imbalance Graph")
plt.show()
Y = dataset['attack'].ravel()#represents attack or normal
data = dataset.values
X = data[:,0:dataset.shape[1]-1]
indices = np.arange(X.shape[0])
np.random.shuffle(indices)#shuffling dataset values
X = X[indices]
Y = Y[indices]
thr = 50
#normalizing dataset values using min max scaler
scaler = MinMaxScaler(feature_range = (0, 1))
X = scaler.fit_transform(X)#normalize train features
print("Normalize Training Features")
print(X)
#now apply smote algorithm on imbalance dataset for balancing
smote = SMOTE()
X, Y = smote.fit_resample(X, Y)
#dataset class labels after applying smote
labels, count = np.unique(Y, return_counts = True)
print("Normal Records After Smote : "+str(count[0]))
print("Attack Records After Smote: "+str(count[1]))
height = count
bars = ['Normal', 'Attack']
y_pos = np.arange(len(bars))
plt.figure(figsize = (4, 3))
plt.bar(y_pos, height)
plt.xticks(y_pos, bars)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size = 0.2)
print("Total records found in dataset = "+str(X.shape[0]))
print("Total features found in dataset= "+str(X.shape[1]))
print("80% dataset for training : "+str(X_train.shape[0]))
print("20% dataset for testing  : "+str(X_test.shape[0]))
#define global variables to save accuracy and other metrics
accuracy = []
precision = []
recall = []
fscore = []
#function to calculate all metrics
def calculateMetrics(algorithm, testY, predict):
    p = precision_score(testY, predict,average='macro') * 100
    r = recall_score(testY, predict,average='macro') * 100
    f = f1_score(testY, predict,average='macro') * 100
    a = accuracy_score(testY,predict)*100
    accuracy.append(a)
    precision.append(p)
    recall.append(r)
    fscore.append(f)
    print(algorithm+" Accuracy  : "+str(a))
    print(algorithm+" Precision : "+str(p))
    print(algorithm+" Recall    : "+str(r))
    print(algorithm+" FSCORE    : "+str(f))
    labels = ['Normal', 'Attack']
    conf_matrix = confusion_matrix(testY, predict)
    fig, axs = plt.subplots(1,2,figsize=(10, 3))
    ax = sns.heatmap(conf_matrix, xticklabels = labels, yticklabels = labels, annot = True, cmap="viridis" ,fmt ="g", ax=axs[0]);
    ax.set_ylim([0,len(labels)])
    axs[0].set_title(algorithm+" Confusion matrix")

    random_probs = [0 for i in range(len(testY))]
```

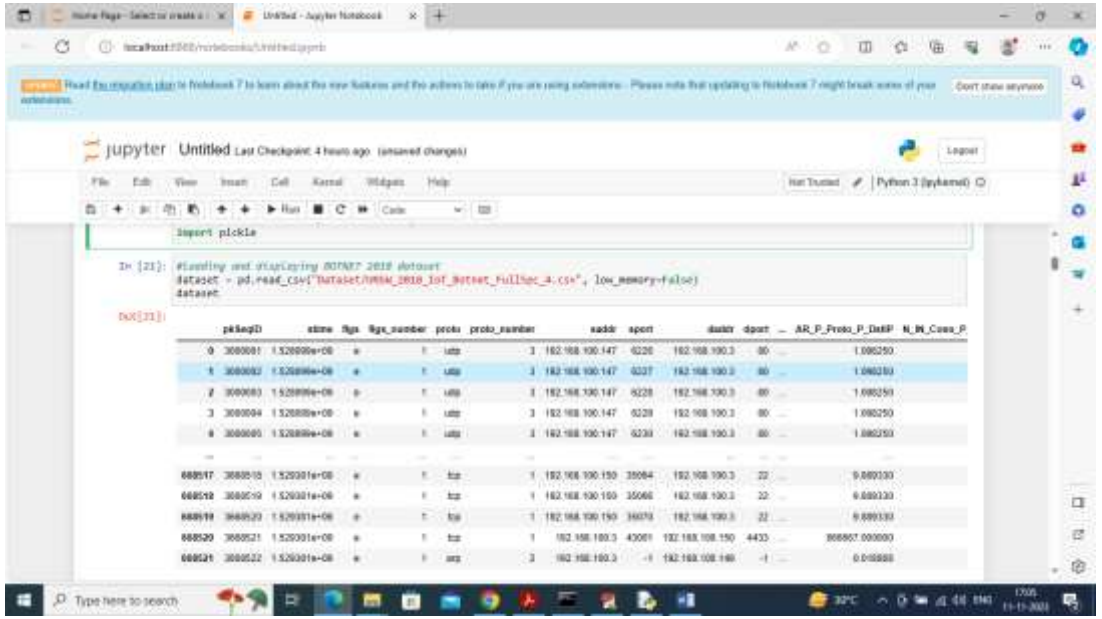
```
p_fpr, p_tpr, _ = roc_curve(testY, random_probs, pos_label=1)
    plt.plot(p_fpr, p_tpr, linestyle='--', color='orange',label="True classes")
    ns_fpr, ns_tpr, _ = roc_curve(testY, predict, pos_label=1)
    axs[1].plot(ns_fpr, ns_tpr, linestyle='--', label='Predicted Classes')
    axs[1].set_title(algorithm+" ROC AUC Curve")
    axs[1].set_xlabel('False Positive Rate')
    axs[1].set_ylabel('True Positive rate')
    plt.show()
#training and evaluating performance of default decision tree algorithm
dt_cls = DecisionTreeClassifier(max_depth=1)
dt_cls.fit(X_train, y_train)#train algorithm using training features and target value
predict =dt_cls.predict(X_test)#perform prediction on test data
#call this function with true and predicted values to calculate accuracy and other metrics
calculateMetrics("Default Decision Tree Algorithm", y_test, predict)
def gaussianProcess(max_depth, min_samples_split, max_features):
    params_dt = {}
    params_dt['max_depth'] = max_depth
    params_dt['min_samples_split'] = min_samples_split
    params_dt['max_features'] = max_features
    scores = cross_val_score(DecisionTreeClassifier(random_state=123, **params_dt),
                             X_train, y_train, cv=5).mean()
    score = scores.mean()
    return score
params_dt = {'max_depth':(5, 10), 'min_samples_split':(0.1, 0.9), 'max_features':(0.1, 0.9)}
dt_bo = BayesianOptimization(gaussianProcess, params_dt, random_state=111)
dt_bo.maximize(init_points=5, n_iter=2)
params_dt = dt_bo.max['params']
```

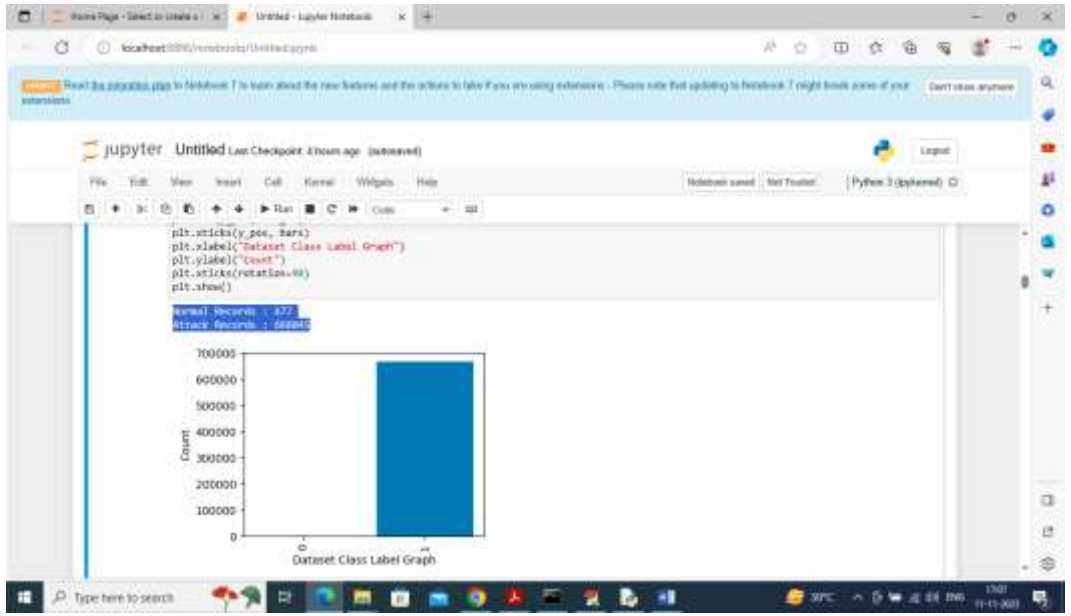
```
print("Best Optimized Parameters Selected by BOGP = "+str(params_dt))
#training and evaluating performance of BOGP optimizied decision tree algorithm
bogp_dt_cls = DecisionTreeClassifier(max_depth=params_dt['max_depth'], max_features=params_dt['max_features'],

min_samples_split=params_dt['min_samples_split'])
bogp_dt_cls.fit(X_train, y_train)#train algorithm using training features and target value
predict = bogp_dt_cls.predict(X_test)#perform prediction on test data
#call this function with true and predicted values to calculate accuracy and other metrics
calculateMetrics("BOGP Optimized Decision Tree", y_test, predict)
#training and evaluating performance of SVM algorithm
svm_cls = svm.SVC()
svm_cls.fit(X_train[0:thr], y_train[0:thr])#train algorithm using training features and target value
predict = svm_cls.predict(X_test) #perform prediction on test data
#call this function with true and predicted values to calculate accuracy and other metrics
calculateMetrics("SVM Algorithm", y_test, predict)
X_train1 = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1, 1))
X_test1 = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1, 1))
y_train1 = to_categorical(y_train)
y_test1 = to_categorical(y_test)
#creating deep learning cnn model object
cnn_model = Sequential()
#defining CNN layer wwith 32 neurons of size 1 X 1 to filter dataset features 32 times
cnn_model.add(Convolution2D(32, (1 , 1), input_shape = (X_train1.shape[1], X_train1.shape[2], X_train1.shape[3]), activation = 'relu'))
#defining maxpool layet to collect relevant filtered features from previous CNN layer
cnn_model.add(MaxPooling2D(pool_size = (1, 1)))
#creating another CNN layer with 16 neurons to optimzed features 16 times
```

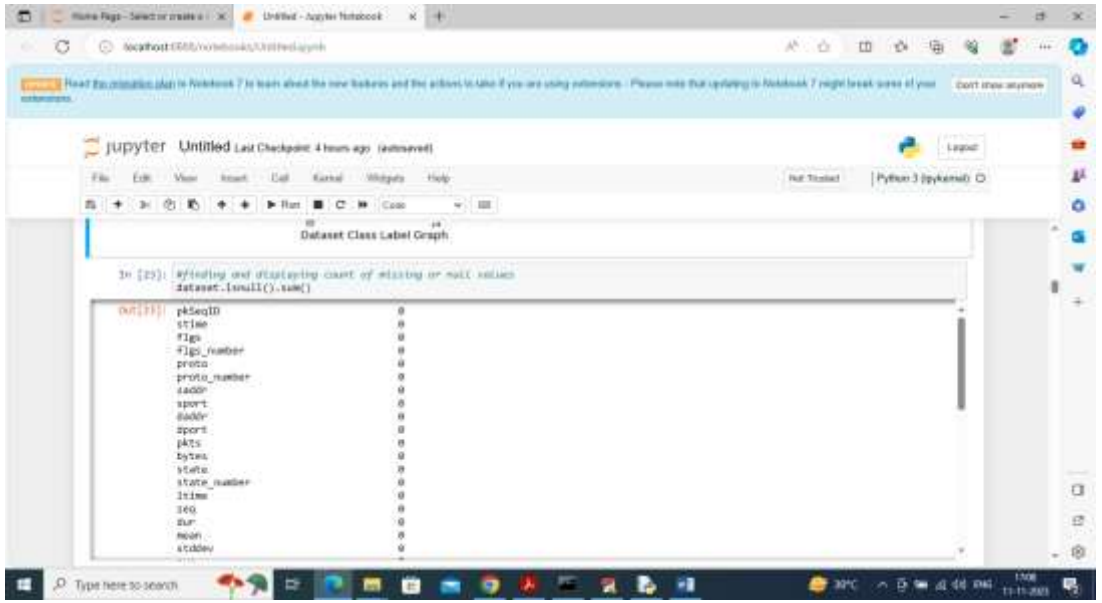
```
cnn_model.add(Convolution2D(16, (1, 1), activation = 'relu'))
cnn_model.add(MaxPooling2D(pool_size = (1, 1)))
#convert multidimension features to single flatten size
cnn_model.add(Flatten())
#define output prediction layer
cnn_model.add(Dense(units = 256, activation = 'relu'))
cnn_model.add(Dense(units = y_train1.shape[1], activation =
'softmax'))
#compile, train and load CNN model
cnn_model.compile(optimizer = 'adam', loss =
'categorical_crossentropy', metrics = ['accuracy'])
if os.path.exists("model/cnn_weights.hdf5") == False:
    model_check_point =
ModelCheckpoint(filepath='model/cnn_weights.hdf5', verbose =
1, save_best_only = True)
    hist = cnn_model.fit(X_train1, y_train1, batch_size = 32,
epochs = 5, validation_data=(X_test1, y_test1),
callbacks=[model_check_point], verbose=1)
    f = open('model/cnn_history.pckl', 'wb')
pickle.dump(hist.history, f)
    f.close()
else:
    cnn_model.load_weights("model/cnn_weights.hdf5")
predict = cnn_model.predict(X_test1)
predict = np.argmax(predict, axis=1)
y_test1 = np.argmax(y_test1, axis=1)
#call this function to calculate accuracy and other metrics
calculateMetrics("Extension CNN", y_test1, predict)
#comparison graph between all algorithms
df = pd.DataFrame([['Default Decision
Tree','Accuracy',accuracy[0]],['Default Decision
Tree','Precision',precision[0]],['Default Decision
Tree','Recall',recall[0]],['Default Decision Tree','FSCORE',fscore[0]],
                ['Optimized Decision
Tree','Accuracy',accuracy[1]],['Optimized Decision
Tree','Precision',precision[1]],['Optimized Decision
Tree','Recall',recall[1]],['Optimized Decision Tree','FSCORE',fscore[1]],
['SVM','Accuracy',accuracy[2]],['SVM','Precision',precision[2]],['SVM','Re
call',recall[2]],['SVM','FSCORE',fscore[2]],
                ['Extension CNN','Accuracy',accuracy[3]],['Extension
CNN','Precision',precision[3]],['Extension
CNN','Recall',recall[3]],['Extension CNN','FSCORE',fscore[3]],
                ],columns=['Parameters','Algorithms','Value'])
df.pivot("Parameters", "Algorithms", "Value").plot(kind='bar', figsize=(6,
3))
plt.title("All Algorithms Performance Graph")
```

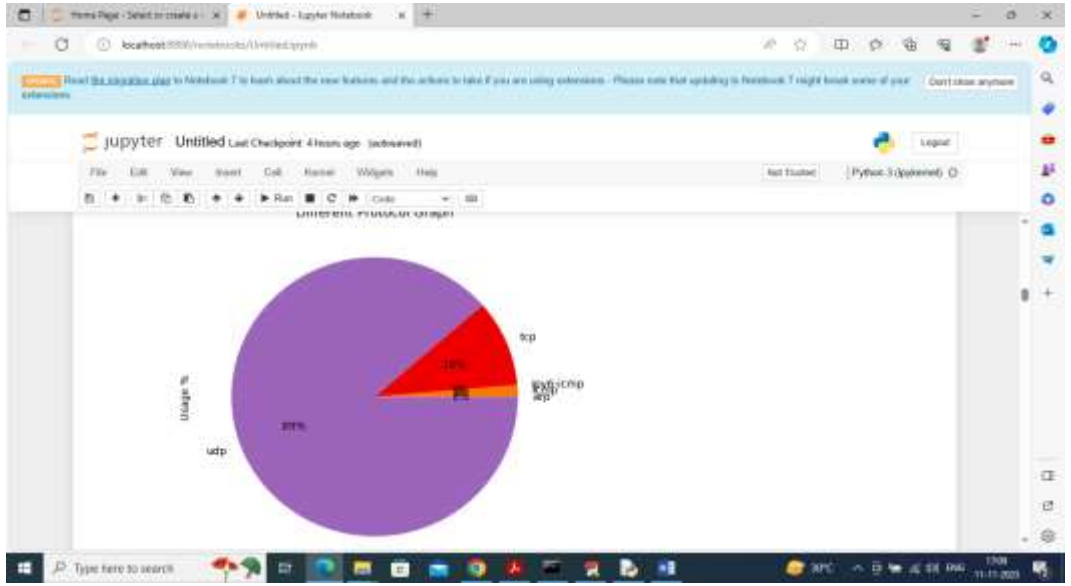
```
plt.show()
#display all algorithm performnace
algorithms = ['Default Decision Tree', 'Optimized Decision Tree', 'SVM', 'Extension CNN']
data = []
for i in range(len(accuracy)):
    data.append([algorithms[i], accuracy[i], precision[i], recall[i], fscore[i]])
data = pd.DataFrame(data, columns=['Algorithm Name', 'Accuracy', 'Precision', 'Recall', 'FSCORE'])
data
```

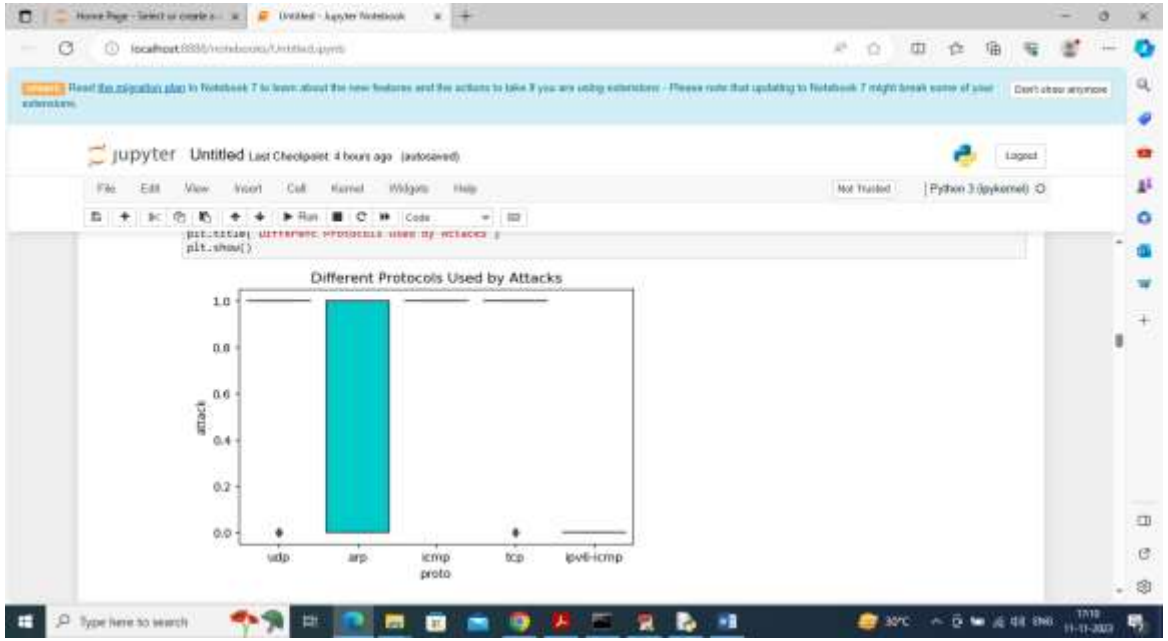
15.SAMPLE OUTPUTS

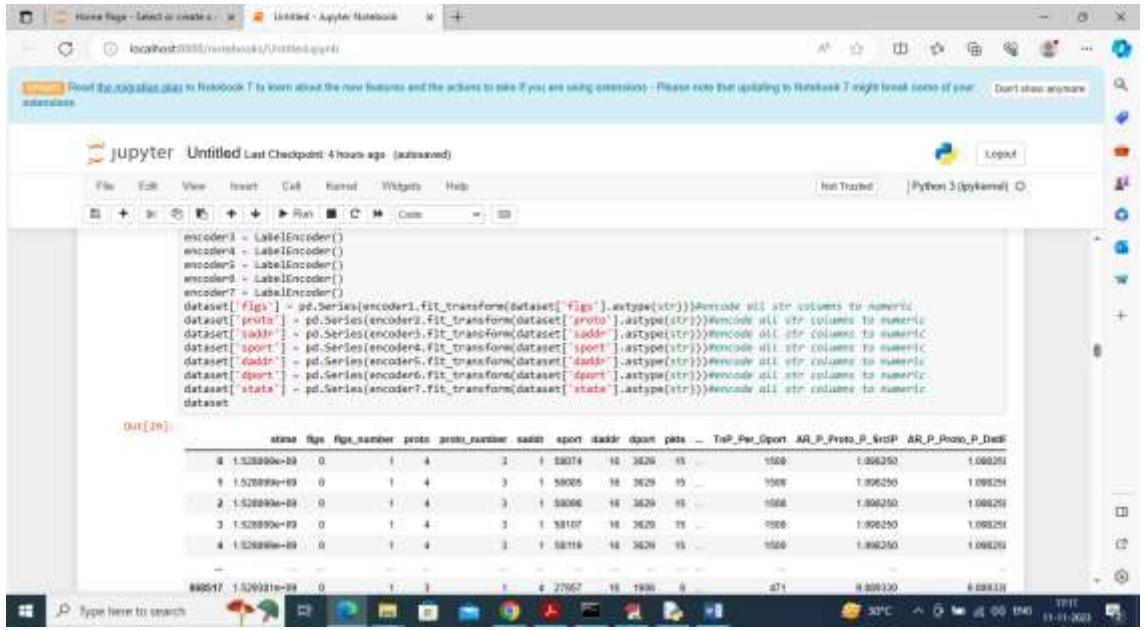


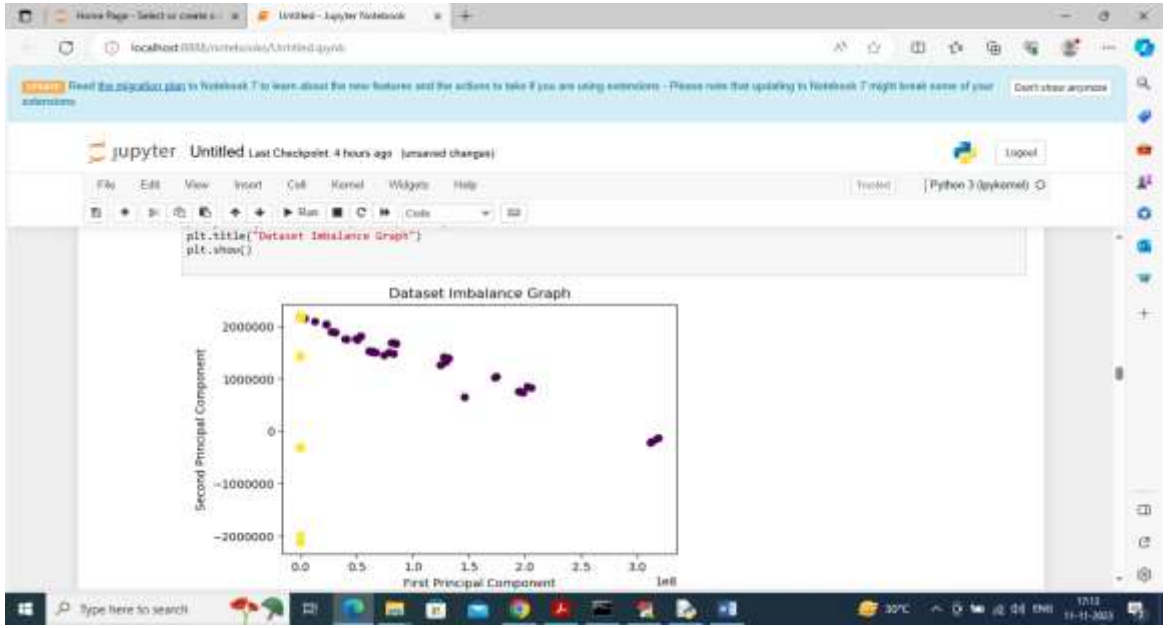


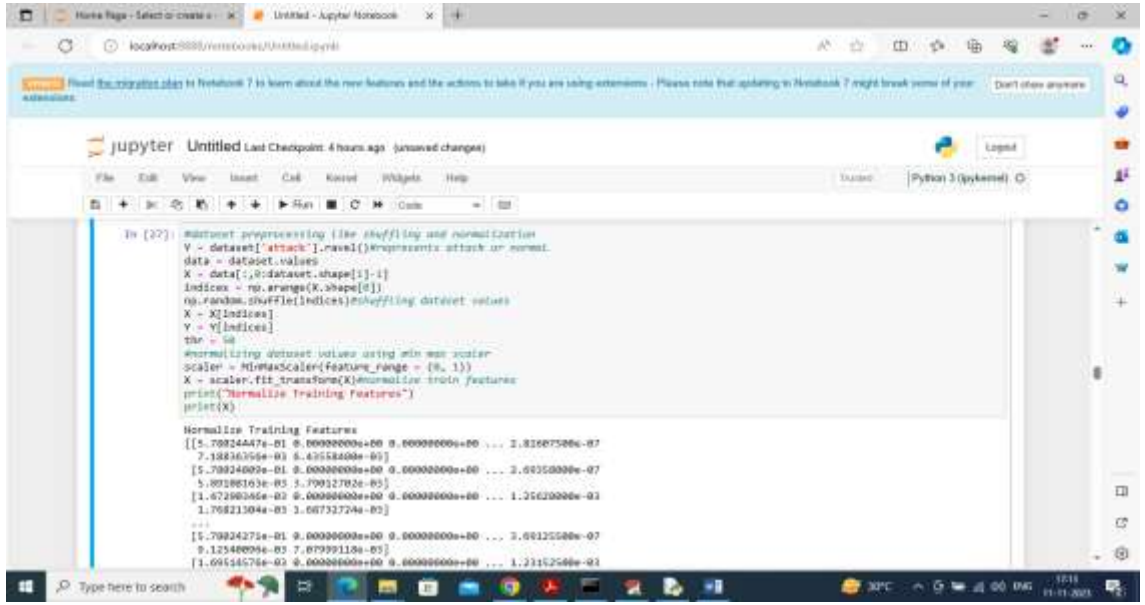





jupyter Untitled Last Checkpoint: 4 hours ago (autosaved)
plt.title('Different Protocols Used by Attacks')
plt.show()
Different Protocols Used by Attacks
attack
proto
udp
arp
icmp
tcp
ipv6-icmp

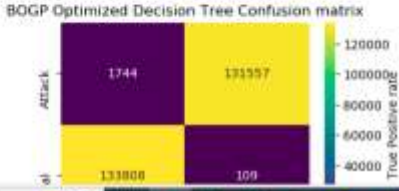

jupyter Untitled Last Checkpoint: 4 hours ago (autosaved)
encoder1 = LabelEncoder()
encoder2 = LabelEncoder()
encoder3 = LabelEncoder()
encoder4 = LabelEncoder()
encoder5 = LabelEncoder()
encoder6 = LabelEncoder()
encoder7 = LabelEncoder()
dataset['flgs'] = pd.Series(encoder1.fit_transform(dataset['flgs'].astype(str)))#encode all str columns to numeric
dataset['proto'] = pd.Series(encoder2.fit_transform(dataset['proto'].astype(str)))#encode all str columns to numeric
dataset['saddr'] = pd.Series(encoder3.fit_transform(dataset['saddr'].astype(str)))#encode all str columns to numeric
dataset['sport'] = pd.Series(encoder4.fit_transform(dataset['sport'].astype(str)))#encode all str columns to numeric
dataset['daddr'] = pd.Series(encoder5.fit_transform(dataset['daddr'].astype(str)))#encode all str columns to numeric
dataset['dport'] = pd.Series(encoder6.fit_transform(dataset['dport'].astype(str)))#encode all str columns to numeric
dataset['state'] = pd.Series(encoder7.fit_transform(dataset['state'].astype(str)))#encode all str columns to numeric
dataset
Out[26]:


jupyter Untitled Last Checkpoint: 4 hours ago (unsaved changes)
plt.title("Dataset Imbalance Graph")
plt.show()
Dataset Imbalance Graph
Second Principal Component
First Principal Component

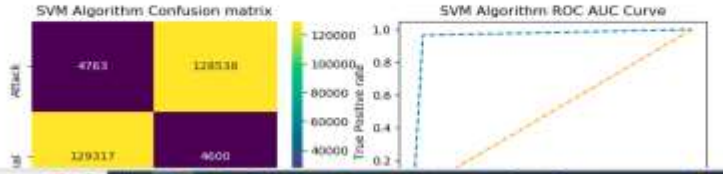

jupyter Untitled Last Checkpoint: 4 hours ago (unsaved changes)
In [27]:
#dataset preprocessing like shuffling and normalization
Y = dataset['attack'].ravel()#represents attack or normal
data = dataset.values
X = data[:,0:dataset.shape[1]-1]
indices = np.arange(X.shape[0])
np.random.shuffle(indices)#shuffling dataset values
X = X[indices]
Y = Y[indices]
thr = 50
#normalizing dataset values using min max scaler
scaler = MinMaxScaler(feature_range = (0, 1))
X = scaler.fit_transform(X)#normalize train features
print("Normalize Training Features")
print(X)
Normalize Training Features

```
In [34]: #giving different parameters as input to bayesian optimization and by using this parameters will process using BOGP to
         #select best parameters
         params_dt = {'max_depth':(5, 10), 'min_samples_split':(0.1, 0.9), 'max_features':(0.1, 0.9)}
         dt_bo = BayesianOptimization(gaussianProcess, params_dt, random_state=111)
         dt_bo.maximize(init_points=5, n_iter=2)
         params_dt = dt_bo.max['params']
         print("Best Optimized Parameters Selected by BOGP = "+str(params_dt))

| iter    | target  | max_depth | max_fe... | min_sa... |
---------------------------------------------------------------
| 1       | 0.9821  | 8.061     | 0.2353    | 0.4488    |
| 2       | 0.9943  | 8.946     | 0.3383    | 0.2193    |
| 3       | 0.9928  | 5.112     | 0.4362    | 0.2909    |
| 4       | 0.9905  | 6.988     | 0.8026    | 0.3982    |
| 5       | 0.9921  | 5.406     | 0.6357    | 0.597     |
| 6       | 0.9995  | 6.584     | 0.9       | 0.1101    |
| 7       | 0.9988  | 6.48      | 0.2044    | 0.1156    |
===============================================================
Best Optimized Parameters Selected by BOGP = {'max_depth': 8.946394594439461, 'max_features': 0.8935694971704703, 'min_samples_s
plit': 0.2909181931201824}

In [35]: #training and evaluating performance of BOGP optimized decision tree algorithm
```

```
In [35]: #training and evaluating performance of BOGP optimized decision tree algorithm
         bogp_dt_cls = DecisionTreeClassifier(max_depth=params_dt['max_depth'], max_features=params_dt['max_features'],
                                              min_samples_split=params_dt['min_samples_split'])
         bogp_dt_cls.fit(X_train, y_train)#train algorithm using training features and target value
         predict = bogp_dt_cls.predict(X_test)#perform prediction on test data
         #call this function with true and predicted values to calculate accuracy and other metrics
         calculateMetrics("BOGP Optimized Decision Tree", y_test, predict)

         BOGP Optimized Decision Tree Accuracy  : 99.30655868990861
         BOGP Optimized Decision Tree Precision : 99.31531182358304
         BOGP Optimized Decision Tree Recall    : 99.30514451699025
         BOGP Optimized Decision Tree FSCORE    : 99.30650947914538
```



```
In [36]: #training and evaluating performance of SVM algorithm
         svm_cls = svm.SVC()
         svm_clx.fit(X_train[0:thr], y_train[0:thr])#train algorithm using training features and target value
         predict = svm_cls.predict(X_test) #perform prediction on test data
         #call this function with true and predicted values to calculate accuracy and other metrics
         calculateMetrics("SVM Algorithm", y_test, predict)

         SVM Algorithm Accuracy  : 96.49611927340224
         SVM Algorithm Precision : 96.49629076627755
         SVM Algorithm Recall    : 96.49500003587244
         SVM Algorithm FSCORE    : 96.49600949533538
```




```
In [37]: #training CNN deep learning algorithm to predict factory maintenance
         #converting dataset shape for CNN compatible format as 4 dimension array
         X_train1 = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1, 1))
         X_test1 = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1, 1))
         y_train1 = to_categorical(y_train)
         y_test1 = to_categorical(y_test)
         #creating deep learning cnn model object
         cnn_model = Sequential()
         #defining CNN layer with 32 neurons of size 1 X 1 to filter dataset features 32 times
         cnn_model.add(Convolution2D(32, (1 , 1), input_shape = (X_train1.shape[1], X_train1.shape[2], X_train1.shape[3]), activation = '
         #defining maxpool layer to collect relevant filtered features from previous CNN layer
         cnn_model.add(MaxPooling2D(pool_size = (1, 1)))
         #creating another CNN layer with 20 neurons to optimized features 16 times
         cnn_model.add(Convolution2D(16, (1, 1), activation = 'relu'))
         #max layer to collect relevant features
         cnn_model.add(MaxPooling2D(pool_size = (1, 1)))
         #convert multidimension features to single flatten size
         cnn_model.add(Flatten())
         #define output prediction layer
         cnn_model.add(Dense(units = 256, activation = 'relu'))
         cnn_model.add(Dense(units = y_train1.shape[1], activation = 'softmax'))
         #compile, train and load CNN model
         cnn_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics = ['accuracy'])
         if os.path.exists("model/cnn_weights.hdf5") == False:
```

16. Conclusion

- Botnet attacks are a major threat to SDN networks.
- Traditional detection systems have limitations in accuracy and scalability.
- The proposed deep learning based system effectively detects botnet attacks.
- Among all tested algorithms, **CNN achieved the highest accuracy**.
- The system improves network security by detecting and mitigating malicious traffic.
- Future work can include **real-time deployment in large scale SDN networks**.

17. References

1.IEEE Transactions on Machine Learning – Botnet Detection in SDN.

2.Research papers on Deep Learning based Intrusion Detection Systems.

3.SDN security research articles.

4.Python Machine Learning documentation.

5.Network Security and Intrusion Detection research publications.